

ДОКУМЕНТАЦИЯ К УЧЕБНОМУ ПРОЕКТУ

Выполнил: Соколов Иван Александрович

Репозиторий: <https://github.com/4ertopolohh/maximaster-practice-2>

Тула, 2026

1. Общая информация о проекте

Проект представляет собой учебное веб-приложение каталога товаров. Приложение включает клиентскую витрину, серверный API и административный интерфейс для управления товарными данными.

Назначение проекта состоит в демонстрации практического применения современных технологий frontend- и backend-разработки при создании каталога товаров с карточками, изображениями, характеристиками и серверной выдачей данных.

Проект решает задачу отображения списка товаров, перехода к детальной странице выбранного товара, получения данных по HTTP API, а также ручного наполнения и редактирования товарного каталога через административный интерфейс Django.

2. Используемые технологии

- Язык программирования backend: Python. В локальной среде установлен Python 3.13.12; в README указано требование Python 3.11 или выше.
- Backend-фреймворк: Django 6.0.4. Версия подтверждается комментариями, созданными Django, в настройках проекта и миграции.
- API: Django REST Framework. Используется приложение `rest_framework`, классы `ListAPIView`, `RetrieveAPIView` и `ModelSerializer`.

- База данных: SQLite. В файле backend/config/settings.py указана база backend/db.sqlite3.
- Работа с изображениями: поле ImageField в модели ProductImage; для такого поля требуется Pillow, что также отражено в инструкции README.
- Язык и платформа frontend: TypeScript, React 19, React DOM 19.
- Маршрутизация frontend: React Router DOM 6.
- Сборка и dev-сервер frontend: Vite 8 с плагином @vitejs/plugin-react.
- Стилизация: Sass/SCSS и CSS Modules.
- Пакетный менеджер frontend: npm; package-lock.json имеет lockfileVersion 3.
- Инструменты контроля качества frontend: ESLint, TypeScript Compiler, Vitest, Testing Library, jsdom.
- Backend-тестирование: django.test.TestCase и стандартный тестовый клиент Django.

3. Требования для запуска проекта

- Git для получения исходного кода проекта.
- Python 3.11 или выше согласно README.md. При установке Django 6.x необходимо учитывать требования конкретной версии Django к версии Python.
- pip и модуль venv для создания изолированного окружения backend.
- Node.js, совместимый с используемой версией Vite. По package-lock.json Vite 8.0.10 требует Node.js ^20.19.0 или >=22.12.0; локальная проверенная версия Node.js: 24.13.1.
- npm для установки frontend-зависимостей; локальная проверенная версия npm: 11.8.0.

4. Инструкция по запуску проекта

Ниже приведена последовательность запуска на новом рабочем месте.

4.1. Получение проекта

```
git clone https://github.com/4ertopolohh/maximaster-practice-2.git
cd maximaster-practice-2
```

4.2. Запуск backend

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install Django djangorestframework Pillow
python manage.py migrate
python manage.py runserver
```

После запуска backend доступен по адресу <http://127.0.0.1:8000/>. API списка товаров доступен по адресу <http://127.0.0.1:8000/api/products/>, детальная карточка товара - по адресу <http://127.0.0.1:8000/api/products/{id}/>, административная панель - по адресу <http://127.0.0.1:8000/admin/>.

4.3. Запуск frontend

Frontend запускается во втором терминале из корня проекта.

```
cd frontend
npm ci
npm run dev
```

Vite сообщает адрес dev-сервера в консоли. Обычно приложение доступно по адресу <http://127.0.0.1:5173/>. В файле `frontend/vite.config.ts` настроено проксирование запросов `/api` и `/media` на <http://127.0.0.1:8000>.

4.4. Сборка и проверка

```
cd frontend  
npm run build  
npm run lint  
npm run test:run
```

Для backend-тестов используется стандартная команда Django из каталога backend.

```
cd backend  
python manage.py test
```

5. Структура проекта

- README.md - основное описание проекта, стек технологий, инструкция запуска и сведения о тестовых данных.
- backend/ - серверная часть на Django: управление проектом, настройки, маршруты, приложение products, миграции и медиафайлы.
- backend/manage.py - стандартная точка входа для административных команд Django.
- backend/config/settings.py - основные настройки Django, включая INSTALLED_APPS, SQLite, MEDIA_URL и MEDIA_ROOT.
- backend/config/urls.py - подключение административной панели и API-маршрутов.
- backend/config/admin.py - кастомный открытый AdminSite, используемый проектом.

- backend/products/models.py - модели Product, ProductImage и ProductCharacteristic.
- backend/products/serializers.py - сериализаторы списка товаров, детальной карточки, изображений и характеристик.
- backend/products/views.py и backend/products/urls.py - обработчики и маршруты API товаров.
- backend/products/tests/ - тесты API и административной панели.
- backend/media/products/ - медиафайлы изображений товаров для backend.
- frontend/ - клиентская часть на React, TypeScript и Vite.
- frontend/package.json и frontend/package-lock.json - скрипты, зависимости и зафиксированное дерево npm-пакетов.
- frontend/vite.config.ts - конфигурация Vite, тестовой среды и прокси до backend.
- frontend/src/App.tsx - маршруты клиентского приложения.
- frontend/src/pages/CatalogPage/ - страница каталога, фильтры сортировки и карточки товаров.
- frontend/src/pages/ProductPage/ - страница детальной карточки товара, базовая информация, галерея и характеристики.
- frontend/src/components/ - переиспользуемые компоненты кнопок и подвала страницы.
- frontend/src/assets/ - изображения и SVG-иконки frontend.
- docs/test-products.txt - тестовые данные для ручного заполнения административной панели.

6. Функционал пользовательского интерфейса

Пользовательский интерфейс реализован как SPA-приложение с маршрутизацией React Router. В файле frontend/src/App.tsx определены маршруты / для каталога, /products/:id для детальной страницы товара и перенаправление неизвестных маршрутов на главную страницу.

- Просмотр каталога товаров, загружаемого запросом GET /api/products/.
- Отображение карточки товара с названием, ценой, статусом наличия и превью-изображением.
- Переход со страницы каталога на детальную страницу по ссылке /products/{id}.
- Сортировка каталога по цене в прямом и обратном порядке.
- Сортировка каталога по алфавиту в прямом и обратном порядке; в коде отдельно обрабатываются русские, английские и прочие первые символы названия.
- Отображение состояний загрузки и ошибок при недоступности API.
- Просмотр детальной карточки товара, загружаемой запросом GET /api/products/{id}/.
- Отображение описания товара, цены, артикула, бренда и статуса наличия.
- Просмотр галереи изображений товара с переключением основного изображения по миниатюрам.
- Отображение дополнительных характеристик товара отдельным блоком.
- Наличие кнопок «Купить» и «В корзину» в карточках и на детальной странице.
- Отображение подвала сайта со ссылками на внешние ресурсы и социальные сети.

7. Функционал административной панели

В проекте реализована административная панель на основе кастомного AdminSite из backend/config/admin.py. Административный сайт подключен по маршруту /admin/. Метод has_permission возвращает True, а метод login перенаправляет на /admin/, поэтому панель в текущей реализации открыта без стандартной авторизации Django.

- В административной панели зарегистрирована модель Product через ProductAdmin.
- В списке товаров отображаются поля title, price, in_stock, sku и brand.
- Для товаров доступны фильтры по наличию и бренду.
- Для товаров доступен поиск по названию, артикулу и бренду.
- Изображения товара редактируются через inline-раздел ProductImageInline с полями image, alt_text и is_preview.
- Характеристики товара редактируются через inline-раздел ProductCharacteristicInline с полями parameter и value.
- Код разрешает просмотр, создание, изменение и удаление товаров, изображений и характеристик через методы has_view_permission, has_add_permission, has_change_permission и has_delete_permission.
- Для изображений реализована проверка, запрещающая выбрать более одного превью-изображения для одного товара.
- В модели ProductImage дополнительно задано ограничение уникальности unique_preview_image_per_product для одного превью на товар.
- Логирование действий администратора для ProductAdmin отключено, поскольку текущая открытая админ-панель работает с анонимными запросами и не может сохранить user_id в журнал Django Admin.